

09-操作系统

操作系统面试题

1. 进程与线程

问：进程和线程的区别？

答：

对比	进程	线程
定义	资源分配的基本单位	CPU 调度的基本单位
资源	独立地址空间	共享进程资源
切换开销	大（需切换页表）	小
通信	IPC（管道、消息队列、共享内存、Socket）	共享变量（需同步）
崩溃影响	不影响其他进程	可能导致整个进程崩溃

问：为什么有了进程还要设计线程？

答：

原因	说明
切换开销	线程切换无需换页表，比进程轻
通信成本	共享地址空间，直接共享变量
资源利用	IO 阻塞时同进程其他线程可运行
多核	多线程可并行利用多核 CPU

进程是资源分配单位（内存、文件描述符、信号等）；线程是调度单位。

问：为什么进程崩溃一般不影响其他进程？

答：- 各进程 独立虚拟地址空间，页表隔离 - 一个进程越界访问不会写到其他进程内存 - 内核通过 MMU 硬件保护 - 崩溃时内核回收该进程 PCB、内存、fd，其他进程继续运行

例外：共享内存、管道等 IPC 需处理对端退出（EPIPE 等）。

问：多线程是不是越多越好？

答：不是。过多线程导致： 1. 上下文切换开销增大，CPU 花在调度而非业务 2. 内存：每线程栈默认 1~8MB 3. 锁竞争加剧 4. 缓存失效：线程在不同核迁移

经验：CPU 密集型 $\approx$ 核数；IO 密集型可更多，但需线程池控制上限。

问：进程有哪些状态？

答：



- 就绪：等 CPU
- 运行：占用 CPU
- 阻塞：等 IO 或事件

问：进程间通信方式？

答：

方式	特点
管道 (Pipe)	半双工，父子进程
命名管道 (FIFO)	无亲缘关系也可
消息队列	按类型存取
共享内存	最快，需同步
信号量	同步与互斥
Socket	跨网络通信

问：信号和信号量有什么区别？

答：

对比	信号 (Signal)	信号量 (Semaphore)
本质	异步通知机制	计数器，用于同步/互斥
用途	通知进程事件发生 (SIGKILL/SIGTERM)	控制资源访问数量
操作	kill() 发送	P(wait)/V(post)
典型	Ctrl+C → SIGINT	生产者-消费者限流

2. 线程同步

问：线程同步的方式？

答： – 互斥锁 (Mutex)：同一时刻只有一个线程持有 – 读写锁：读共享、写独占 – 信号量 (Semaphore)：控制并发数量 – 条件变量 (Condition)：等待某个条件成立 – 自旋锁：忙等待，适合临界区很短

问：用户态和内核态的区别？

答：

对比	用户态	内核态
权限	低，不能访问硬件	高，可执行特权指令
切换	–	系统调用、中断、异常时切换
开销	小	大（上下文切换）

3. 死锁

问：死锁的四个必要条件？

答： 1. 互斥：资源不能共享 2. 占有且等待：持有资源同时等待其他资源 3. 不可抢占：资源不能被强制剥夺 4. 循环等待：存在等待环路

处理策略：预防（破坏条件）、避免（银行家算法）、检测+恢复、鸵鸟策略。

4. 内存管理

问：分页和分段的区别？

答：

对比	分页	分段
单位	固定大小页框	逻辑段（代码段、数据段）
碎片	内部碎片	外部碎片
地址	一维（页号+偏移）	二维（段号+偏移）
目的	提高内存利用率	满足逻辑需求

问：页面置换算法？

答：

算法	说明
FIFO	先进先出，简单但有 Belady 异常
LRU	最近最久未使用，性能好但实现复杂
LFU	最不经常使用
OPT	最优（理论，不可实现）
Clock	LRU 近似，环形队列

问：虚拟内存是什么？

答：将磁盘空间作为内存扩展，程序可使用超过物理内存的地址空间。

实现：请求分页/分段，缺页中断时从磁盘调入页面。

好处：提高内存利用率、支持多道程序、逻辑地址空间大于物理内存。

问：虚拟地址如何转换为物理地址？

答：分页机制（现代 OS 主流）：

```
虚拟地址 = 页号 + 页内偏移
            ↓ 查页表（MMU 硬件完成）
物理地址 = 页框号 + 页内偏移
```

多级页表：减少页表占用（如 x86-64 四级页表）。

TLB：页表缓存，加速地址翻译；进程切换时 TLB 刷新。

缺页中断：页表项无效或不在内存 → 内核从磁盘调入页或分配页框。

问：程序的内存布局是怎样的？堆和栈的区别？

答：

```
高地址
  栈 Stack（向下增长）← 局部变量、函数参数、返回地址
  ↓
  内存映射区（mmap、共享库）
  堆 Heap（向上增长）← malloc/new 动态分配
  BSS 段 ← 未初始化全局变量
  Data 段 ← 已初始化全局/静态变量
  Text 段 ← 代码（只读）
低地址
```

对比	栈	堆
分配	编译器自动 / 函数调用	malloc/new 手动
释放	函数返回自动	free/delete 或 GC
速度	快（移动栈指针）	慢（需管理空闲块）
大小	有限（MB 级）	较大
线程	每线程独立栈	进程共享堆

5. 调度算法

问：常见进程调度算法？

答：

算法	说明
FCFS	先来先服务，简单但短作业等待长
SJF	短作业优先，平均等待最短
时间片轮转	公平，适合分时系统
优先级调度	优先级高的先执行
多级反馈队列	综合多种策略

6. 文件系统

问：硬链接和软链接的区别？

答：

对比	硬链接	软链接（符号链接）
本质	同一 inode 的别名	独立文件，存目标路径
跨分区	否	是
源删除	仍可访问	失效（悬空）
inode	共享	独立

7. IO 多路复用

问：select、poll、epoll 的区别？

答：

对比	select	poll	epoll
数据结构	bitmap	链表	红黑树 + 就绪链表
最大连接	1024	无限制	无限制
效率	O(n) 遍历	O(n) 遍历	O(1) 事件通知
拷贝	每次需拷贝 fd 集合	同	mmap 共享
触发	水平触发	水平触发	水平/边缘触发

epoll 适合：大量连接、少量活跃的高并发场景（如 Redis、Nginx）。

问：五种 IO 模型是什么？

答：

模型	说明
阻塞 IO	read 阻塞直到数据就绪
非阻塞 IO	read 立即返回，轮询（忙等）
IO 多路复用	select/poll/epoll 监听多 fd，就绪再 read
信号驱动 IO	SIGIO 通知数据就绪
异步 IO（AIO）	OS 完成 read 后通知（Proactor）

Java NIO = 非阻塞 + 多路复用（Reactor）；Netty 基于 epoll。

问：服务器处理高并发有哪些方式？

答：

方式	说明
多进程	每连接一进程，Apache prefork，开销大
多线程	每连接/线程池，Tomcat
IO 多路复用	单/少线程 + epoll，Redis、Nginx
协程	用户态调度，Go、Java Virtual Thread
异步 IO	IOCP、io_uring

问：Reactor 和 Proactor 的区别？

答：– **Reactor**：同步非阻塞 IO，应用负责 read/write（Java NIO、Netty） – **Proactor**：异步 IO，OS 完成 read/write 后通知应用（Windows IOCP）

问：什么是零拷贝？

答：减少数据在用户态和内核态之间的拷贝次数。

方式：– `mmap`：内核缓冲区映射到用户空间 – `sendfile`：文件 → 内核缓冲区 → Socket，不经用户态 – `splice`：管道转发

传统 4 次拷贝：磁盘 → 内核 → 用户 → 内核 → Socket。`sendfile` 减少到 2 次。

8. 中断

问：什么是中断？硬中断和软中断？

答：– **中断**：CPU 暂停当前程序，转去处理紧急事件 – **硬中断**：由硬件触发（键盘、网卡），优先级高 – **软中断**：由内核触发（系统调用、异常），优先级低

问：中断的处理流程是什么？

答：

1. CPU 完成当前指令，保存 PC、寄存器到内核栈
2. 根据中断向量表跳转到中断处理程序（内核态）
3. 保存完整上下文
4. 执行 ISR (Interrupt Service Routine)
5. 恢复上下文，返回被中断的程序

硬中断：可打断软中断；软中断不可打断硬中断。

9. 设备管理

问：键盘敲入 A 字母，操作系统期间发生了什么？

答：

1. 键盘控制器产生硬件中断
2. CPU 切换到内核态，执行中断处理程序
3. 驱动读取扫描码，转换为 ASCII 'A'
4. 放入键盘缓冲区
5. 唤醒等待输入的进程
6. 应用程序 `read()` 系统调用读取 'A'

10. 进程线程协程

问：进程、线程、协程的区别？

答：

对比	进程	线程	协程
调度	操作系统	操作系统	用户态程序（或运行时）
切换开销	最大（页表、TLB）	中等	最小（仅保存寄存器）
内存	独立地址空间	共享进程堆、共享库	共享线程栈或独立栈
通信	IPC	共享变量	无锁，直接传值
崩溃	隔离	同进程线程受影响	同线程内影响

协程适用：高并发 IO 密集（Go goroutine、Java Virtual Thread、Python asyncio），在 IO 阻塞时主动让出 CPU，单线程支撑万级连接。

问：用户态到内核态切换的过程和开销？

答：触发场景：系统调用（read/write）、中断（时钟/网卡）、异常（缺页/除零）。

1. 保存用户态上下文（寄存器、PC、SP）到内核栈
 2. 切换到内核栈，权限级别提升（CPL 3 → 0）
 3. 执行内核处理程序（如 read 从磁盘读数据）
 4. 恢复用户态上下文，返回用户程序

开销来源：– 上下文切换（寄存器、栈） – TLB 刷新（若切换进程） – CPU 缓存失效 – 内核路径执行

优化：减少系统调用（批量 read/write）、IO 多路复用、零拷贝、协程减少线程切换。

11. 内存管理深入

问：什么是写时复制（Copy-On-Write）？

答：fork() 创建子进程时，父子共享同一物理页，页表标记为只读。

1. fork() 后父子映射相同物理页，引用计数 +1
 2. 任一方尝试写入 → 缺页异常 → 内核复制一份新物理页
 3. 修改方页表指向新页，另一方仍用旧页

优点：fork 极快，只有真正写入时才复制。

应用：fork + exec 启动进程、Redis RDB 持久化（BGSAVE fork 子进程写磁盘）、Java ForkJoinPool 部分场景。

问：fork() 会复制哪些东西？

答：

复制/共享	内容
COW 共享	代码段、数据段、堆、栈（物理页）
独立复制	PID、PCB、页表（结构复制，物理页 COW）
继承	打开的文件描述符、信号处理、工作目录
不继承	父进程锁、定时器（部分）

exec() 后：替换整个地址空间，只保留 fd（若未设 FD_CLOEXEC）。

问：brk 和 mmap 分配内存的区别？

答：

对比	brk	mmap
位置	堆顶（data segment）向上扩展	独立虚拟内存区域
释放	不立即归还 OS，可复用	立即归还 OS
大小	适合小内存（<128KB 通常）	适合大内存映射
场景	malloc 小对象	大对象、文件映射、共享内存

malloc 策略：小对象用 brk 池化管理，大对象直接 mmap；free 后小对象进空闲链表，大对象 munmap 归还。

问：LRU 页面置换算法如何实现？

答：Least Recently Used，淘汰最久未访问的页。

实现方式：

方式	数据结构	复杂度
理论 LRU	时间戳/链表	O(1) 但硬件开销大
实际近似	Clock 算法（环形链表 + 访问位）	O(n) 扫描
Linux	双链表（活跃/非活跃）+ 多级 LRU	综合性能

Clock 算法：指针扫环形队列，访问位=1 则置 0 并跳过，=0 则淘汰；是 LRU 的近似。

Belady 异常：FIFO 增加页框数反而缺页率上升，LRU 和 OPT 不会有此问题。

问：银行家算法是什么？

答：死锁避免算法，在分配资源前预判是否会导致不安全状态。

数据结构:

- Available: 可用资源向量
- Max: 各进程最大需求
- Allocation: 已分配
- Need = Max - Allocation

步骤:

1. 检查请求 $\leq$ Available 且 $\leq$ Need
2. 试探分配, 检查是否所有进程都能完成 (安全序列)
3. 安全则分配, 否则进程等待

局限: 需预知最大资源需求, 进程数和资源类型有限, 实际系统多用预防 (破坏四条件) 或检测+恢复。

问: 操作系统内存不足时会发生什么?

答: 1. 回收缓存: 释放 page cache、buffer cache 2. **swap 换出**: 将不活跃页换到磁盘 swap 分区 3. **OOM Killer**: Linux 按 oom_score 杀死进程 (dmesg | grep oom) 4. **malloc 失败**: 返回 NULL 或抛 bad_alloc

避免 OOM: 合理设置 JVM/Redis maxmemory、监控 swap 使用率、限制容器 memory limit。

12. IO 多路复用深入

问: epoll 的 ET 和 LT 模式区别?

答:

模式	触发条件	特点
LT (水平触发)	缓冲区有数据就一直通知	默认模式, 安全, 编程简单
ET (边缘触发)	状态变化时通知一次	高效, 必须一次读完 (循环 read 到 EAGAIN)

ET 注意事项: - 必须设非阻塞 IO - 写事件也要边缘处理 (写不完要监听 EPOLLOUT) - Nginx、Netty 默认 ET

LT vs ET 类比: LT 像电平信号, 高电平持续触发; ET 像边沿信号, 上升沿触发一次。

问: Reactor 模型详解?

答:

Reactor (同步非阻塞 IO + IO 多路复用):

- 1. Main Reactor (Acceptor): 监听连接, accept 新连接
- 2. Sub Reactor (Processor): epoll_wait 监听已连接 fd 的读写事件
- 3. Handler: 事件就绪后, 由线程池执行业务逻辑

单 Reactor 单线程: Redis 6.0 前主线程

单 Reactor 多线程: 业务丢线程池

主从 Reactor 多线程: Netty、Muduo, accept 和 IO 分离

Proactor 对比: OS 异步 IO 完成后通知 (Windows IOCP), 应用无需自己 read/write。

问: 为什么 Redis 和 Nginx 这么快?

答:

因素	Redis	Nginx
单线程/多进程	单线程 IO 多路复用 (6.0 后 IO 线程可选)	多 Worker 进程, 每进程 epoll
内存	全内存操作	静态资源 sendfile 零拷贝
协议	简单 RESP, 无复杂解析	HTTP 解析高效
数据结构	底层优化 (SDS、跳表、压缩列表)	事件驱动, 无阻塞
网络	epoll ET + 非阻塞	epoll LT/ET + 连接池

共同核心: IO 多路复用 + 非阻塞 + 零拷贝 + 避免上下文切换 (Redis 单线程无锁)。

13. 调度与同步深入

问: 上下文切换时保存了哪些内容?

答: - CPU 寄存器: PC、SP、通用寄存器 - 进程/线程状态: 运行 → 就绪/阻塞 - 内存管理: 切换进程时切换页表、刷新 TLB - 内核栈: 每个线程独立内核栈

线程切换比进程切换轻量: 共享地址空间, 无需切换页表。

问: 哲学家就餐问题如何解决死锁?

答: 经典同步问题, 5 人5 筷子, 每人需左右两根才能吃。

解法: 1. 破坏循环等待: 按序拿筷子 (先拿小编号) 2. 破坏占有且等待: 一次性拿两根 3. 破坏互斥: 允许三人同时吃 (限制并发) 4. 检测+恢复: 超时放下筷子重试

对应实际: 锁排序、一次性获取所有锁、tryLock 超时。

图解加深

专题	图解章节	要点
进程与线程	进程与线程	PCB、上下文切换
虚拟内存	内存管理	分页、缺页中断、COW
进程调度	进程调度	时间片、优先级
锁	互斥与同步	互斥锁、信号量
IO 多路复用	Socket	select/poll/epoll 对比图
零拷贝	文件系统	sendfile 流程图

大厂追问

1. **fork 后父子谁先运行？** 不确定，由调度器决定；可通过写时复制和写顺序观察。
2. **一个进程最多能创建多少线程？** 受虚拟内存（栈空间）、`ulimit -u`、PID 上限限制，32 位约几百，64 位可达数万。
3. **epoll 的惊群问题？** 多进程 listen 同一 fd，事件到来全部唤醒，仅一个 accept；解决：`SO_REUSEPORT` 每进程独立 listen fd。
4. **Redis 单线程如何处理阻塞命令？** KEYS、FLUSHALL 等会阻塞主线程，生产环境用 SCAN 替代；持久化 fork 子进程。
5. **用户态线程（协程）能否利用多核？** 单线程协程不能，需多线程 + 每线程跑协程调度器（Go M:N 模型）。
6. **缺页中断发生在什么时候？** 访问不在物理内存的页（首次访问、被换出、COW），内核从磁盘调入或复制页。
7. **进程上下文包含什么？** CPU 寄存器（PC/SP/通用寄存器）、进程状态、内核栈、页表基址、打开文件表、信号掩码等。
8. **乐观锁和悲观锁在 OS 层面的体现？** 悲观：mutex/signal 先加锁再操作；乐观：CAS 无锁，失败重试（如 atomic 指令）。